ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ГОРОДА МОСКВЫ
«Колледж связи № 54» имени П.М. Вострухина

ОТЧЕТ О ВЫПОЛНЕНИИ

по дисциплине

“Внедрение и поддержка компьютерных систем”

Практической работы №6

«Обработка ошибок и защита соединений»

Выполнил: ст.гр. 3-ИСП11-43

ФИО: Юхимчук Евгений Викторович

Специальность: 09.02.07

Информационные системы и программирование

Проверил: преподаватель Кандыбаева

Москва

2026

Практическая работа №6. Обработка ошибок и защита соединений

Ссылка на репозиторий: https://github.com/gangdexyuhi/client-server-practice
Папка работы: practice_06_client_server/

Цель работы

Научиться защищать работу с базой данных конструкцией try/except/finally, откатывать транзакцию при ошибке с помощью rollback() и гарантированно закрывать соединение, чтобы приложение не падало и база не блокировалась.

Используемые технологии

- Python 3.13: исключения, try/except/finally
- sqlite3: commit(), rollback(), close(), исключения sqlite3.Error и sqlite3.IntegrityError
- Flask 3.1, шаблон index.html с блоком ошибки

Структура проекта

```
src/
├── init_db.py, check_db.py, crud_console.py
├── app.py                # защищённые чтение и добавление
├── app.db
├── requirements.txt
└── templates/
    └── index.html
```

Описание реализации

Соединение открывается до блока try, чтобы в finally всегда было что закрыть. Внутри try выполняются execute и commit. В except sqlite3.Error вызывается rollback(), ошибка выводится в консоль сервера, а пользователю возвращается страница с понятным сообщением. Блок finally закрывает соединение при любом исходе. Функция чтения fetch_all_tasks() защищена так же и при ошибке возвращает пустой список.

```
def fetch_all_tasks():
    """Чтение задач. При ошибке БД возвращает пустой список, а не роняет
приложение."""
```

```
    conn = get_connection()
    try:
        cursor = conn.cursor()
        cursor.execute("SELECT id, title, status FROM tasks ORDER BY id")
        return cursor.fetchall()      # finally выполнится до возврата значения
    except sqlite3.Error as e:
        print(f"[DB ERROR] {e}")
        return []
    finally:
        conn.close()


@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        title = request.form.get('title', '').strip()

        if not title:
            return render_template(
                'index.html',
                items=fetch_all_tasks(),
                error='Название задачи не может быть пустым.'
            )

        # соединение открываем ДО try, чтобы в finally было что закрывать
        conn = get_connection()
        try:
            cursor = conn.cursor()
            cursor.execute("INSERT INTO tasks (title) VALUES (?)", (title,))
            conn.commit()
        except sqlite3.Error as e:
            conn.rollback()           # отменяем незавершённую транзакцию
            print(f"[DB ERROR] {e}")  # пишем ошибку в консоль сервера
            return render_template(
                'index.html',
                items=fetch_all_tasks(),
                error='Не удалось сохранить задачу. Попробуйте ещё раз.'
            )
        finally:
            conn.close()              # закрывается при любом исходе

        return redirect(url_for('index'))

    return render_template('index.html', items=fetch_all_tasks())
```

Ход работы и результаты запуска

1. Проверил, что приложение работает в обычном режиме.

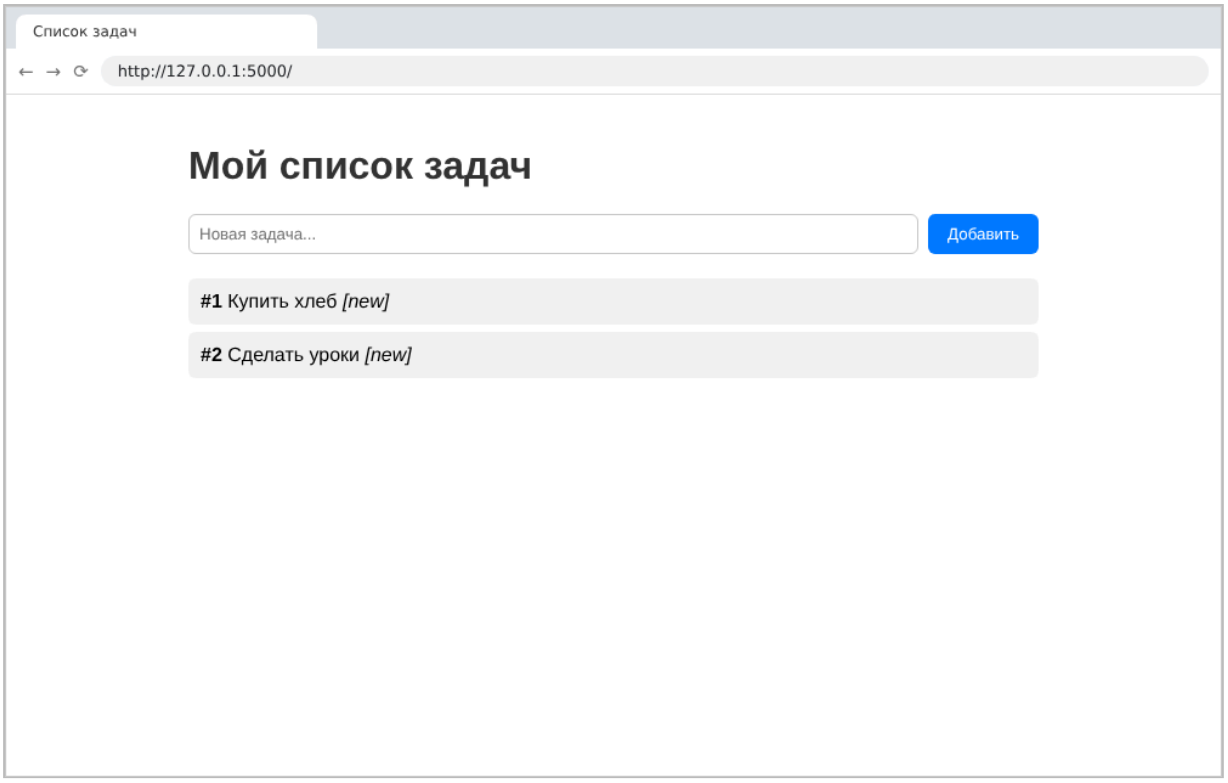


Рисунок 1 — Приложение в обычном режиме

2. Эксперимент 1: временно заменил параметр (title,) на (None,), чтобы нарушить ограничение NOT NULL. При отправке формы возникла ошибка IntegrityError, сработал except: выполнен rollback, в консоль выведено сообщение, а пользователь увидел «Не удалось сохранить задачу». Сервер продолжил работать.

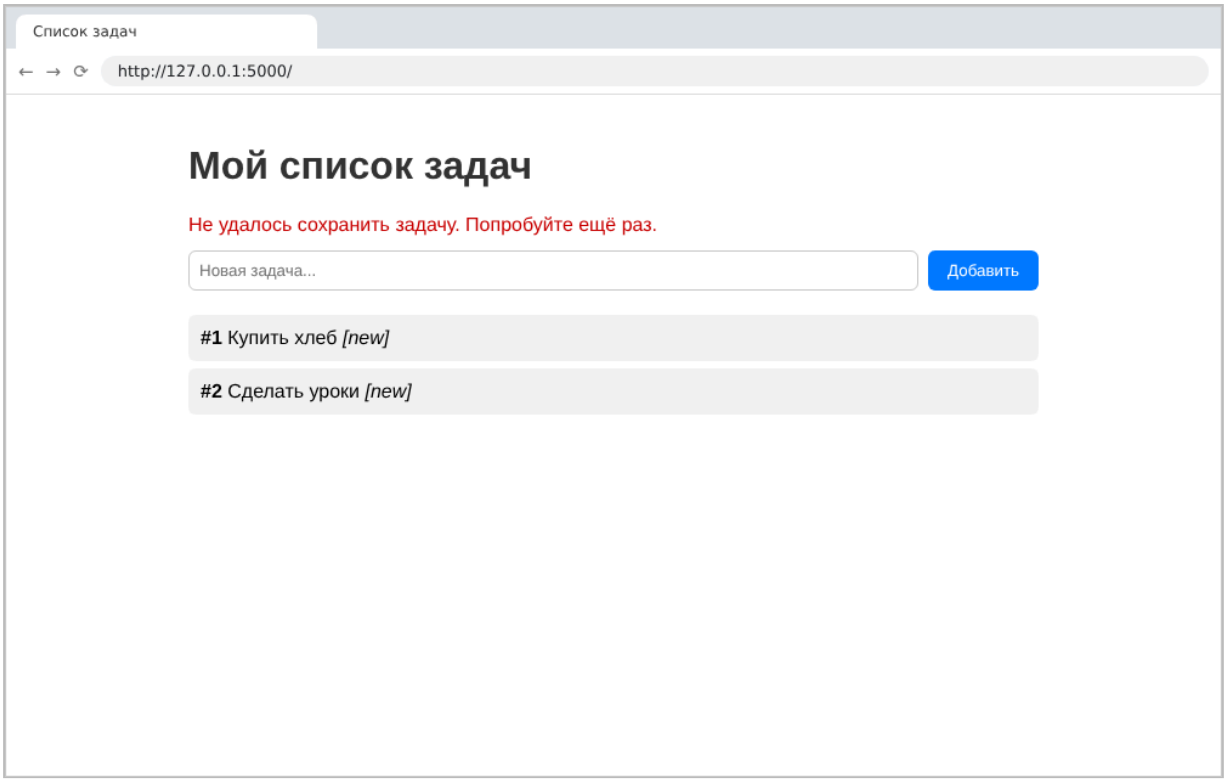


Рисунок 2 — Сообщение пользователю при ошибке базы данных

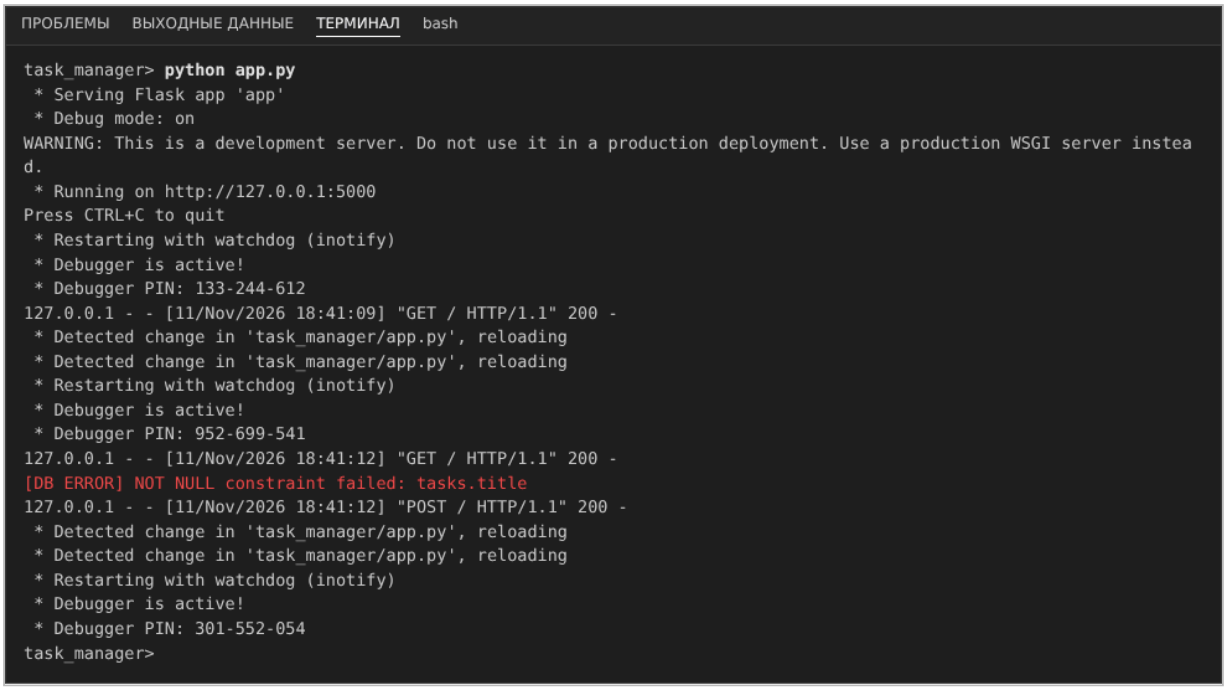


Рисунок 3 — Журнал сервера: [DB ERROR] NOT NULL constraint failed

3. После эксперимента проверил базу через crud_console.py во втором терминале: лишних строк нет (rollback сработал), а база не заблокирована — новую задачу удалось добавить, так как finally закрыл соединение.

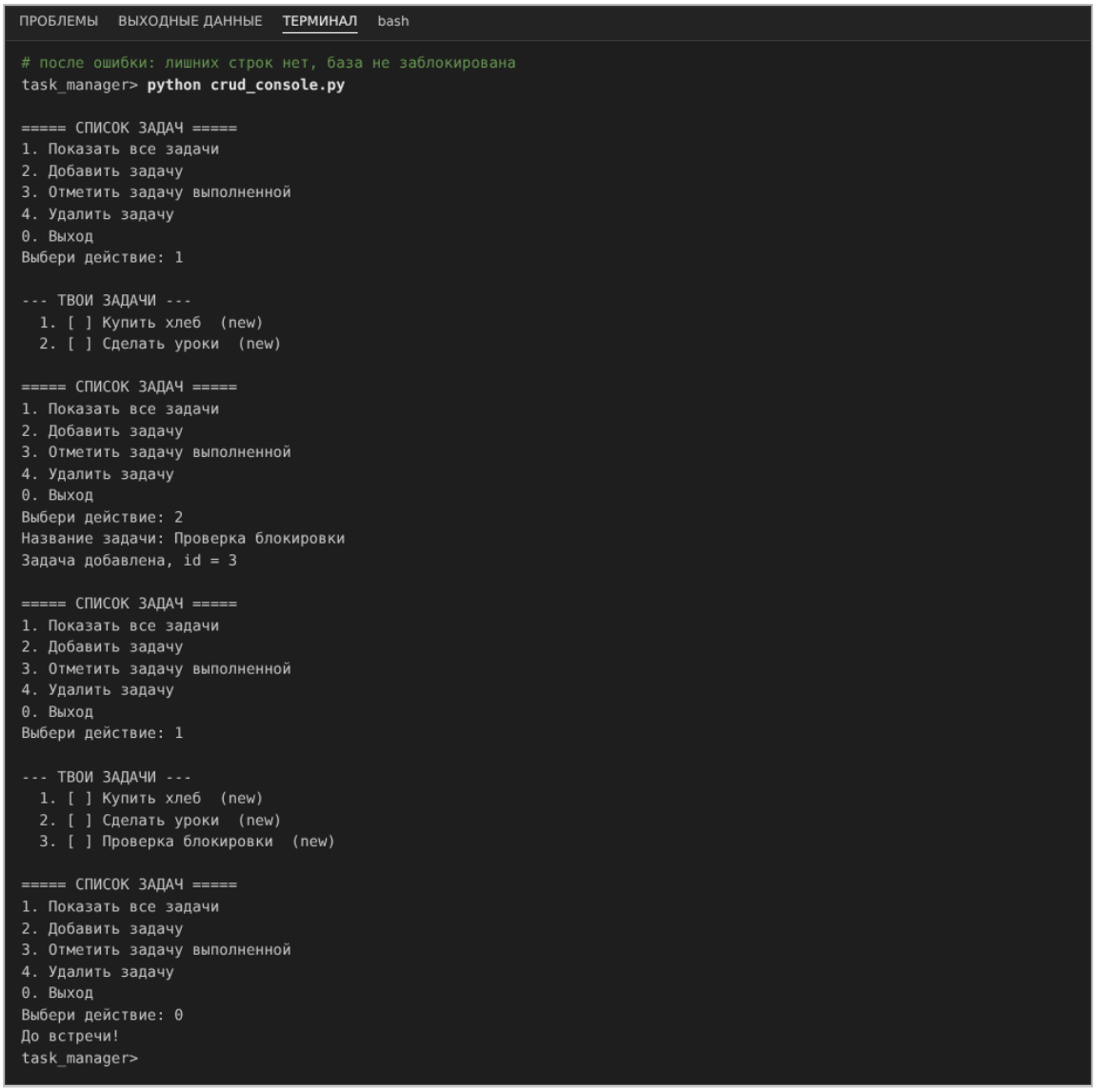


Рисунок 4 — Проверка базы после ошибки: нет мусорных строк и блокировки

4. Эксперимент 3: добавил деление на ноль между execute и commit. Исключение ZeroDivisionError не относится к sqlite3.Error, поэтому наш except его не поймал и rollback не вызвался; в режиме отладки Flask показал страницу с трассировкой. Блок finally при этом всё равно закрыл соединение. Вывод: в except нужно перехватывать все исключения, которые могут возникнуть в try, либо использовать except Exception.

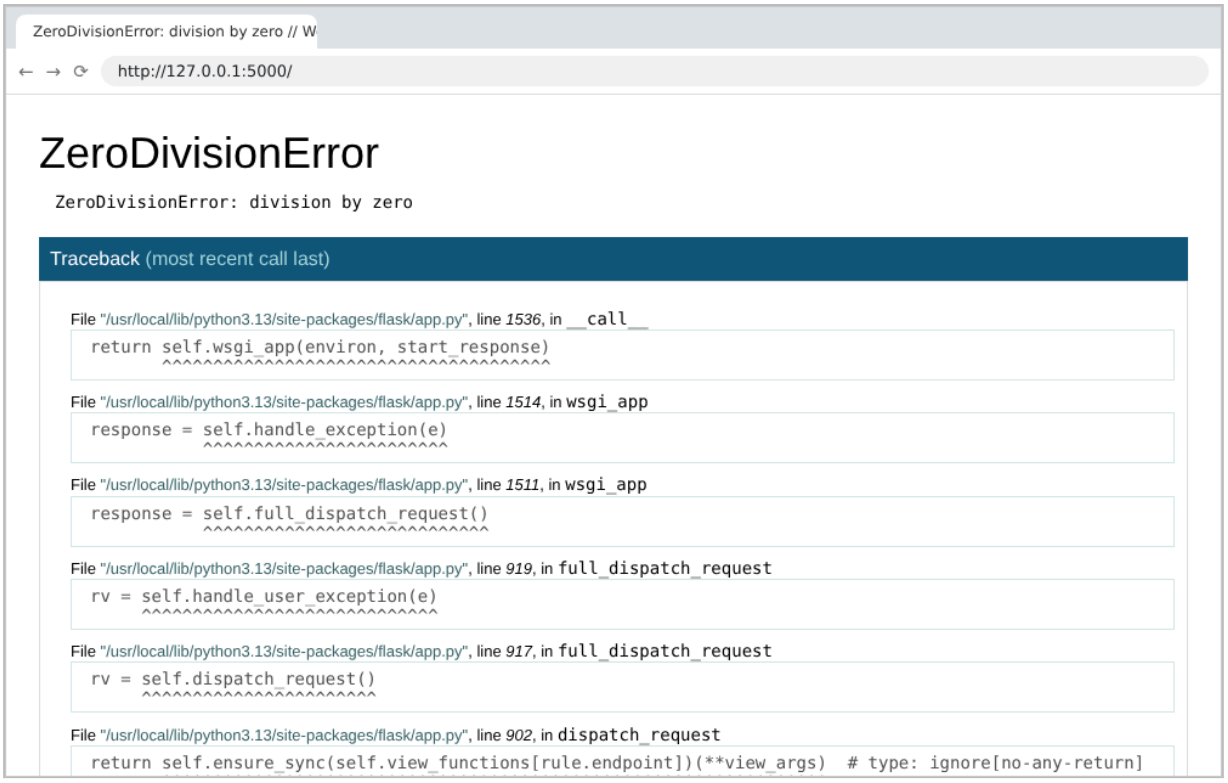


Рисунок 5 — Неперехваченное исключение ZeroDivisionError

5. Вернул код в исходное состояние — приложение снова работает, задача из эксперимента 3 в базу не попала.

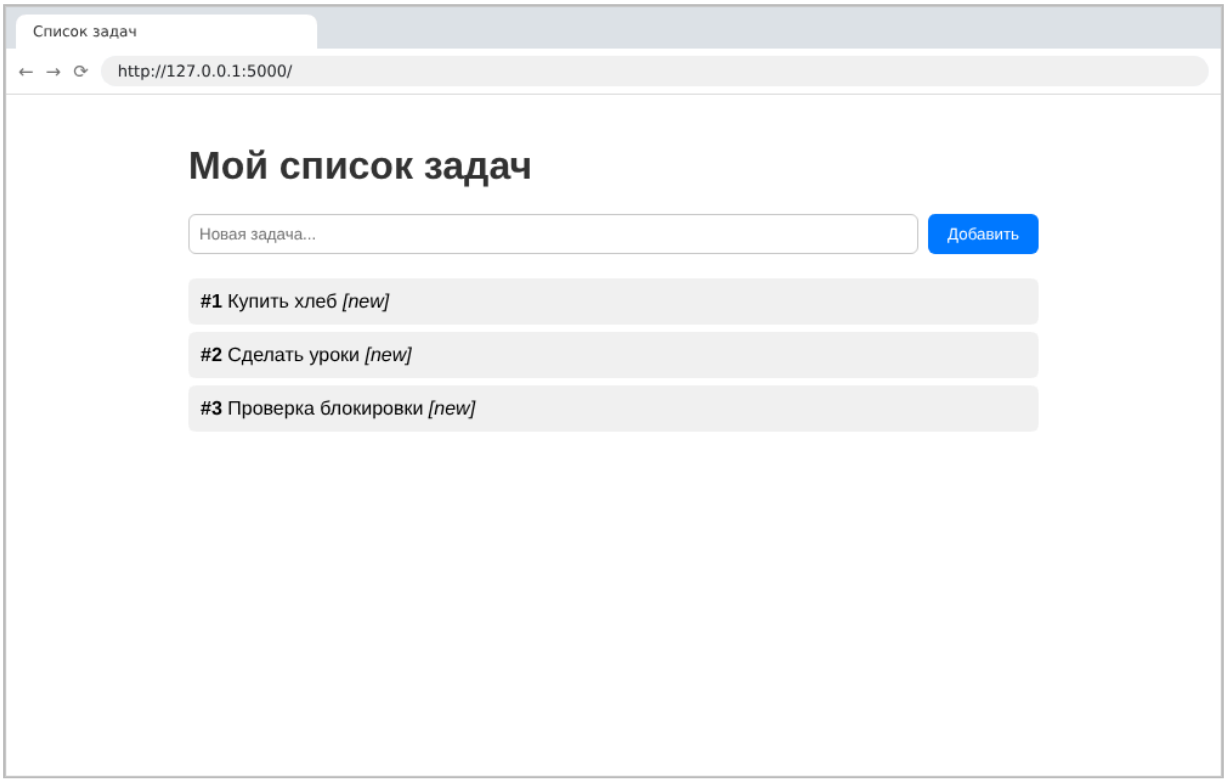


Рисунок 6 — Приложение после возврата исходного кода

Примеры запросов и ответов

Таблица 1 — Примеры HTTP-запросов и ответов сервера

Запрос	**Код ответа**	**Заголовок Location**	**Результат**
POST / form: title='Купить молоко'	302 FOUND	/	Задача сохранена, редирект на список
POST / form: title=''	200 OK	—	Страница с сообщением о пустом названии
POST / form: title='Задача с ошибкой' (эксперимент с None)	200 OK	—	rollback, страница «Не удалось сохранить задачу»

Ответы на контрольные вопросы

1. Почему conn.close() нельзя оставлять просто в конце функции, а нужно помещать в finally?

Если в execute или commit возникнет исключение, выполнение функции прервётся, и строка с close() в конце не выполнится. Соединение останется открытым, файл базы может оказаться заблокированным. Блок finally выполняется всегда — и при успехе, и при ошибке, и при return.

2. Что делает conn.rollback() и в каком случае он вызывается?

rollback() отменяет все изменения текущей незафиксированной транзакции и возвращает базу в состояние до её начала. Он вызывается в блоке except, когда во время работы с базой произошла ошибка.

3. Почему conn нужно открывать до try, а не внутри?

Если открыть соединение внутри try и ошибка произойдёт ещё до присваивания conn, то в finally переменной conn не будет и возникнет UnboundLocalError. Открытие до try гарантирует, что в finally есть объект соединения.

4. Чем except sqlite3.Error лучше, чем except: без указания типа?

Голый except перехватывает вообще все исключения, включая KeyboardInterrupt и SystemExit, а также ошибки в нашем собственном коде, которые лучше увидеть. Перехват sqlite3.Error обрабатывает только ошибки базы данных, остальные проблемы не маскируются.

5. Что произойдёт с данными, если execute прошёл, а commit упал?

Изменения не зафиксируются в файле. Транзакция останется незавершённой, и её нужно явно откатить через rollback() в except, чтобы в базе не осталось частично записанных данных и соединение не держало блокировку.

6. Зачем в веб-приложении возвращать пользователю понятное сообщение вместо стектрейса?

Стектрейс непонятен обычному пользователю и раскрывает внутреннее устройство приложения (код, пути к файлам, структуру базы), что небезопасно. Понятное сообщение сообщает, что произошло, и позволяет продолжить работу.

7. Как проверить, что rollback действительно откатил транзакцию?

Сравнить содержимое таблицы до и после ошибки: вывести список задач через crud_console.py или выполнить SELECT COUNT(*). В эксперименте количество задач не изменилось — значит, незавершённая вставка была отменена.

Вывод

В ходе работы я защитил операции с базой данных конструкцией try/except/finally с rollback(). На экспериментах я убедился, что при ошибке базы приложение не падает, пользователь видит понятное сообщение, в базе не остаётся мусорных строк, а соединение закрывается и база не блокируется. Также я понял, что except должен перехватывать все исключения, которые реально могут возникнуть в блоке try.